



Nozibul Islam

Posted on Sep 23, 2024 • Edited on Oct 4, 2024



6



7



7



7



7

Interview Questions and Answers in Hoisting

#webdev #javascript #interview #programming

1. What is hoisting in JavaScript?

Answer: Hoisting is the process during the creation phase of the execution context where memory is allocated for variables and functions. During this process, the memory for variables is allocated, and the variables are assigned the value undefined. For functions, the entire function definition is stored at a specific address in memory, and a reference to it is placed on the stack in that particular execution context.

2. What is the difference between variable hoisting and function hoisting?

Answer: Variable hoisting occurs when a variable's declaration is moved to the top of its scope. On the other hand, function hoisting happens when the entire function, including its body, is moved to the top of its scope.

3. How does hoisting work in JavaScript?

Answer: Hoisting is a JavaScript mechanism where variable and function declarations are moved to the top of their scope before the code execution begins. Because of hoisting, we can use variables before they are declared.

- **Variable hoisting:** When a variable is declared, it's moved to the top of its scope, and we can use that variable even before its declaration in the code.
- **Function hoisting:** When a function is hoisted, the entire function body is moved to the top of its scope. This means we can call the function even before its declaration in the code.

4. What is the value of a variable that is declared but not initialized?

Answer: If you declare a variable but do not assign it a value, the variable is considered to have no value. JavaScript moves the variable declaration to the top during hoisting, but the variable is not yet assigned a value. The variable is assigned undefined, meaning the variable has not received any value yet.

5. What happens if you declare a variable twice in the same scope?

Answer: If you declare the same variable twice in the same scope, the second declaration will overwrite the first one, and the value of the first declaration will be replaced by the second. This happens because JavaScript stores the first declaration in memory, and when it encounters the second one, it overwrites the previous value.

6. Can you hoist a variable without using the var keyword?

Answer: Yes, in JavaScript, you can hoist variables using let or const instead of var. While var has been traditionally used, it has some issues related to scope, which is why it is recommended to use let or const.

Example with let:

```
let myVariable = 42;  
console.log(myVariable); // Output: 42
```

Example with const:

```
const pi = 3.14159;  
console.log(pi); // Output: 3.14159
```

Variables declared with let and const have block-level scope, meaning they are confined to the block (such as a function, loop, or statement) where they are defined. let allows reassignment, while const is used for constants and cannot be reassigned.

7. What is the difference between a function declaration and a function expression in terms of hoisting?

Answer: A function declaration is hoisted to the top of its scope, whereas a function expression is not. This means that you can call a function declared with a function declaration before its definition, but you cannot do the same with a function expression.

8. What is the scope of a hoisted variable?

Answer: The scope of a hoisted variable depends on where it is declared. If the variable is declared inside a function, its scope is limited to that function, meaning it cannot be accessed outside of it. If a variable is declared outside of any function, it has a global scope, meaning it can be accessed from anywhere in the code.

9. How can you prevent hoisting in JavaScript?

Answer: Hoisting cannot be entirely prevented in JavaScript, but you can write your code in a way that avoids potential issues caused by it. One way to do this is by declaring all variables and functions at the top of their scope.

In the code examples above, all variables and functions are declared at the top of their block using `var` or `let`. This ensures that variables and functions are hoisted within their block and appear at the top of their scope.

This method makes the code flow and variable values more predictable, which helps reduce variability and potential bugs.

10. What are the potential pitfalls of hoisting in JavaScript?

Answer: Hoisting can lead to potential issues if you don't fully understand how it works, resulting in unexpected behaviors. For example, if you declare a variable inside a block statement and try to access it outside the block, the variable might be hoisted to the top of the function or global scope, which may not be what you intended.

- **Block scope and hoisting:** Variables declared inside a block may still be hoisted to the top of their scope.
- **Debugging difficulty:** Hoisting can make it hard to debug because the value or variable you are trying to access may not behave as expected.
- **Function definition quirks:** Functions are hoisted to the top of their scope, but you may not always use them in the way you expect.

To deal with these hoisting issues, it is important to declare your variables and functions clearly and understand how hoisting works within your code.

11. Can hoisting change the order of execution in JavaScript?

Answer: Yes, hoisting in JavaScript moves variable and function declarations to the top of their respective scopes before the code execution begins. This can change the expected flow of code execution.

With hoisting, you can declare a variable and use it before it is actually defined, which might change the expected order of code execution and lead to unintended outcomes. For instance, if you do not define functions at the beginning of the script and later use them, they will be hoisted to the top, potentially creating unexpected flow and behavior in the code.

12. What is the difference between hoisting in var and let declarations?

Answer: var declarations are hoisted to the top of their scope, whereas let declarations are hoisted to the top of their block. This means that variables declared with let cannot be accessed outside their block.

Let's look at an example to clarify this:

```
function example() {  
    console.log(x); // undefined  
    var x = 5;  
    console.log(x); // 5  
}  
example();  
console.log(x); // ReferenceError: x is not defined
```

Here, `var x = 5;` is hoisted to the top of the function scope. However, when `console.log(x)` is called before the declaration, the value is `undefined` due to hoisting.

Now let's look at an example with let:

```
function example() {  
    console.log(y); // ReferenceError: y is not defined  
    let y = 10;  
    console.log(y); // 10  
}  
example();  
console.log(y); // ReferenceError: y is not defined
```

Here, `let y = 10;` is hoisted to the top of its `block scope`, but attempting to access it before its declaration results in a `ReferenceError`.

In summary, `var` variables are hoisted to the top of the function scope, while `let` variables are hoisted to the top of their block.

13. What happens if you try to access a hoisted variable before it is declared?

Answer: If you try to access a hoisted variable before it is declared, its value will be `undefined`.

14. What is the hoisting behavior of arrow functions in JavaScript?

Answer: Arrow functions are not hoisted in JavaScript, so you cannot call an arrow function before it is defined.

```
// This will work
function regularFunction() {
  console.log("This is a regular function");
}
regularFunction(); // "This is a regular function"

// This will not work
arrowFunction(); // TypeError: arrowFunction is not a function
const arrowFunction = () => {
  console.log("This is an arrow function");
};
```

Here, `regularFunction` is hoisted and can be called before its definition, but `arrowFunction` is not hoisted, so calling it before its definition will result in an error.

15. Can hoisting occur within a function?

Answer: Yes, hoisting can occur within a function. This means that variables or functions declared inside a function are hoisted to the top of that function's scope.

16. What is the scope chain in JavaScript?

Answer: In JavaScript, the scope chain is the hierarchy of scopes used by the JavaScript engine to look for the value of a variable. The scope chain includes the current function's scope, outer function scopes, and the `global scope`.

The scope chain works as a step-by-step process for finding the value of a variable. If a variable or function is declared inside a `function`, the JavaScript engine first looks within the scope of that function. If the value is not found there, it searches the outer function and continues this process until it reaches the global scope.

In this way, JavaScript uses the scope chain to find variable values and updates the chain according to where the variable is defined.

17. What will be the output of the following code snippet?

Answer:

```
console.log(a);  
var a = 10;  
Output: undefined
```

18. What will be the output of the following code snippet?

Answer:

ReferenceError: a is not defined

In this case, `a` is declared using `let`, which means it is not hoisted to the top of its scope. Since `a` is not defined before the `console.log()` statement, a reference error is thrown.

19. What will be the output of the following code snippet?

Answer:

```
var a = 5;  
(function() {  
    console.log(a); // undefined  
    var a = 10;  
})();  
Output: undefined
```

To understand this process fully, let's break it down step-by-step:

In the first line, `var a = 5;` creates a variable `a` with the value of 5 in the global scope.

Then, an immediately invoked function expression (IIFE) is defined and called.

Inside the function, `console.log(a);` tries to print the value of `a`. However, because `var a = 10;` is declared within the function, the local variable `a` is hoisted to the out of the function's scope, but its value has not yet been assigned, so it prints `undefined`.

After that, `var a = 10;` creates a new `a` variable in the local scope with the value of 10.

Thus, the first `console.log(a);` prints `undefined` due to hoisting, and after the declaration, the value of `a` is updated to `10`.

20. What will be the output of the following code snippet?

Answer:

```
function test() {  
  console.log(a);  
  var a = 10;  
  console.log(a);  
}  
test();  
Output:  
undefined  
10
```

The function `test()` is defined with two `console.log()` statements:

In the first `console.log(a);` the variable `a` is declared within the function but has not yet been assigned a value, so it prints `undefined`.

In the second `console.log(a);` the variable `a` is assigned the value `10`, which is printed as `10`.

Thus, when the function is called, the first `console.log(a);` prints `undefined`, and the second `console.log(a);` prints `10`.

Connect with me on LinkedIn:

I regularly share insights on JavaScript, TypeScript, Node.js, React, Next.js, data structures, algorithms, web development, and much more. Whether you're looking to enhance your skills or collaborate on exciting topics, I'd love to connect and grow with you.

Follow me: [Nozibul Islam](#)

```
✖ ▶ Error: Text content does not match server-rendered HTML.  
  at checkForUnmatchedText (react-dom.development.js:9647:1)  
  at diffHydratedProperties (react-dom.development.js:10310:1)  
  at hydrateInstance (react-dom.development.js:11306:1)  
  at prepareToHydrateHostInstance (react-dom.development.js:12507:1)  
  at completeWork (react-dom.development.js:22181:1)  
  at completeUnitOfWork (react-dom.development.js:26596:1)  
  at performUnitOfWork (react-dom.development.js:26568:1)  
  at workLoopSync (react-dom.development.js:26466:1)  
  at renderRootSync (react-dom.development.js:26434:1)  
  at performConcurrentWorkOnRoot (react-dom.development.js:25704:1)  
  at workLoop (scheduler.development.js:266:1)  
  at flushWork (scheduler.development.js:239:1)  
  at MessagePort.performWorkUntilDeadline (scheduler.development.js:113:1)  
  
③ ▶ Error: Hydration failed because the initial UI does not match  
  what was rendered on the server.  
  at throwOnHydrationMismatch (react-dom.development.js:12507:1)  
  at tryToClaimNextHydratableInstance (react-dom.development.js:10310:1)  
  at updateHostComponent (react-dom.development.js:19902:1)  
  at beginWork (react-dom.development.js:21618:1)  
  at beginWork$1 (react-dom.development.js:27426:1)  
  at performUnitOfWork (react-dom.development.js:26557:1)  
  at workLoopSync (react-dom.development.js:26466:1)  
  at renderRootSync (react-dom.development.js:26434:1)  
  at performConcurrentWorkOnRoot (react-dom.development.js:25704:1)
```

[Read More](#)

Top comments (0)

[Code of Conduct](#) • [Report abuse](#)

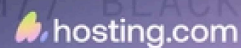


Hosting.com PROMOTED

...



GET MANAGED WORDPRESS FOR **\$1.00** FOR THE FIRST MONTH



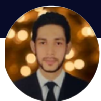
[Your next WordPress project? Deployed for \\$1.](#)

This Black Friday, hosting.com powered by Rocket.net gives you fully Managed WordPress Hosting for just \$1 in your first month.

No setup headaches. No caching chaos. Just pure speed with Cloudflare Enterprise CDN, advanced security, and automatic updates.

Whether it is a side project or your next client build, you will be live in minutes. Grab the \$1 deal and start building.

[Learn more](#)



Nozibul Islam

I am a Full-Stack Developer specialized Front-end Developer. Passionate about algorithms, data structures, and coding challenges & always ready to face new challenges.

LOCATION

Dhaka, Bangladesh

EDUCATION

B.Sc. in Computer Science Engineering

PRONOUNS

Mr.

WORK

Currently, I'm focused on problem-solving and learning DSA while freelancing.

JOINED

Aug 24, 2024

More from [Nozibul Islam](#)

[Boost]

[#twopointer](#) [#codingpattern](#) [#programming](#) [#leetcode](#)

Two Pointer coding pattern

[#twopointer](#) [#codingpattern](#) [#programming](#) [#leetcode](#)

15 Rendering patterns for Web Application

[#webdev](#) [#javascript](#) [#programming](#) [#react](#)



Postmark

PROMOTED



Find out why “Postmark = ”

(Their words, not ours)



“Postmark blows every other transactional email product out of the water.”

Kyle Racki, CEO

Josh Sommers, Lead Developer

[Get started free](#)



[See why Postmark = !\[\]\(4d25d87d94191bbe34f0046ad604e903_img.jpg\)](#)

[Get started free](#)